DAT 5668
Module 1 Notes
The Agent Loop

Salih Tutun, PhD
Olin Business School
Washington University in St. Louis

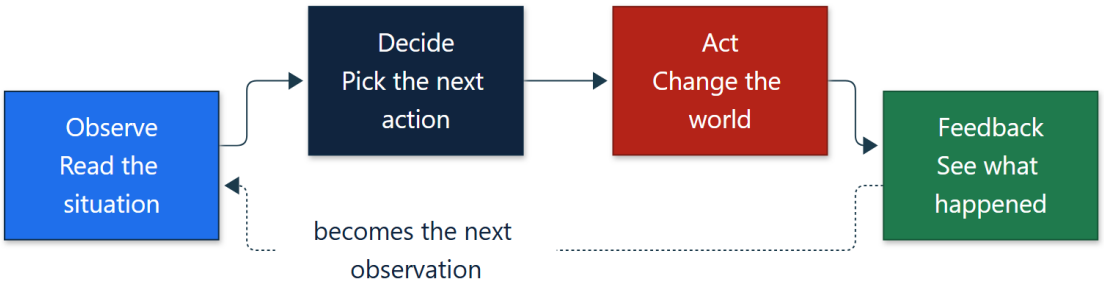
Fall 2026

Abstract

These notes are the reading for Module 1. Session 1 teaches the idea in the room. Session 2 is where you build the tool-using agent. Open `Session-01.ipynb` and `Session-02.ipynb` beside this document. The figures are the same ones in the notebooks. There is no recording to watch first.

1 What an agent is

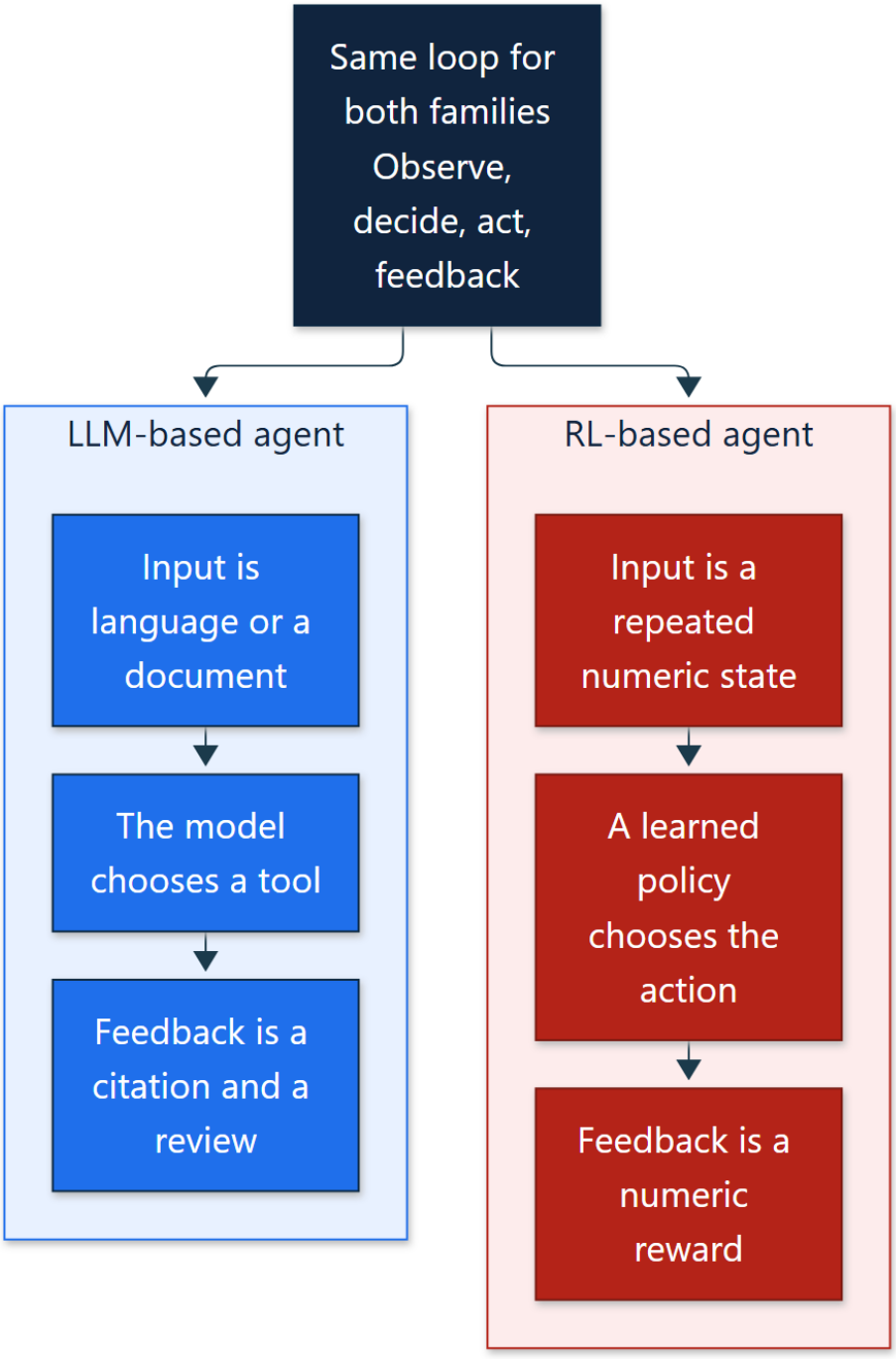
An agent acts toward a goal in an environment. The definition is the loop, not the brand of the model.



Step	Meaning	Refund desk
Observe	Read the situation	Ticket text, order age, order total
Decide	Pick the next action	Look up the policy, then approve, refuse, or escalate
Act	Change something outside the model	Post the refund or open a case
Feedback	See whether the action helped	The amount came from the order record, and a person can audit the tools

A chatbot that only writes a polite paragraph is not yet an agent. It never took an action the desk can audit, and it has no feedback tied to the order. The refund amount has to come from the system of record. The model does not invent a dollar figure.

2 Two families, one loop



An LLM-based agent and an RL-based agent answer the same managerial question with different machinery.

- An **LLM-based agent** reads language or a document, chooses a tool, and is checked by a citation and a review.

- An **RL-based agent** reads a repeated numeric state, chooses an action from a learned value, and is checked by a reward.

Using the wrong family is a design error. A chat reply is a poor inventory controller: the state is stock, the action is an order quantity, and the feedback is a number that repeats. A deep Q-network is a poor contract reviewer: the input is unstructured language, and the action is a tool call plus a reason a person can check.

Session 1 runs both families on the same tickets so you can see where they agree and where they do not. The RL family is trained later, starting in Session 4. Session 1 only shows a small learned table.

3 The same three tickets

The case is instructional. It is not data from a firm. The policy the business will stand behind is short:

- A damaged item may be refunded within 30 days.
- An order older than 30 days is refused unless a manager escalates.
- "Changed my mind" is not a damage claim. The policy tool refuses it.
- The refund amount is the order total.

Ticket	Text	Age (days)	Total
A	Item arrived cracked	12	$80
B	Changed my mind	10	$40
C	Screen is cracked	45	$200

3.1 The language-model side

The decide step may name only three tools: `lookup_policy`, `lookup_amount`, and `finish`. It may not invent a fourth tool, and it may not invent the dollar amount. `lookup_amount` reads the order total. `finish` returns refund, refuse, or escalate.

In class the decide step is a local stand-in, so the notebook runs with no API key. If a Gemini key is set, that same step can ask Gemini which tool to call next. The tools still run in the notebook. A missing key, a refused call, or a tool name outside the schema falls back to the stand-in. Do not paste a firm secret into the ticket text.

On these three tickets the stand-in refunds A, refuses B, and refuses C. Ticket C is damaged, but it is outside the 30-day window.

3.2 The reinforcement-learning side

The RL agent does not read the sentence. A small mapping turns the ticket into a state: fresh damage, stale damage, or other. It then picks the action with the largest learned value. Actions are refund, refuse, and escalate.

Ticket B is the point of the session. The language-model agent refuses it, because the policy tool says a change of mind is not eligible. The RL agent has no damage rule for that sentence, so the state is "other," and the learned table escalates. That disagreement is a design fact. It is not a bug in the code.

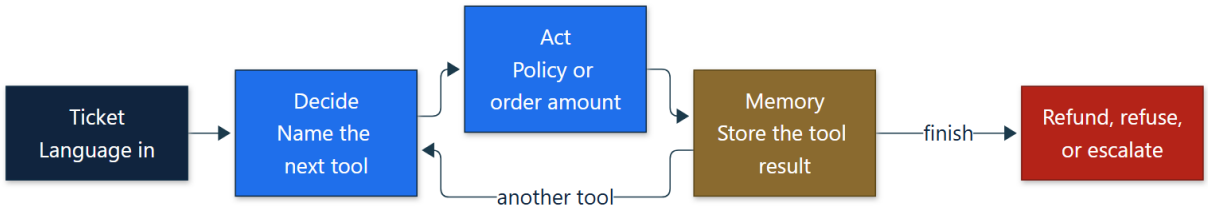
4 What to say to a manager

Ask these five questions before anyone builds the agent. They are the same questions for both families.

1. **Goal.** What decision, if automated, would change a KPI this quarter?
2. **Data.** Is the input a document or ticket, or a repeated numeric state?
3. **Actions.** What may the system do, and what is forbidden?
4. **Feedback.** Is the signal a citation and a human rating, or a numeric reward every step?
5. **Human checkpoint.** Which action is too costly to take without a person?

Inside the policy window, a damaged item can be auto-approved. "Other" should be seen by a person. Neither family should override the 30-day window.

5 Session 2: from a chatbot to an agent



Session 2 is the practice session. You build the language-model side. The business rule is the same: refund a damaged item only when the order is 30 days old or newer, and take the amount from the order record.

A chatbot cell in the notebook refunds an invented $999. That cell is the failure mode. Your `decide` function has to beat it.

1. Observe the order with `lookup_order`. An unknown id escalates.
2. Observe the rule with `lookup_policy` on the order note.
3. Refund only when the reason is eligible and the age is inside the window. Otherwise refuse.
4. If the order is missing, escalate and call `open_case`.
5. The refund amount equals the order total. Do not hard-code 999.

The function returns `action`, `amount`, `reason`, and `case_id`. Groups in the room may be 1–3 students. The homework after class is individual.

5.1 The homework rule

Homework 1 adds one rule to the Session 2 agent. If the order total is above $150, do not auto-refund. Escalate and open a case. The rest of the policy stays as it was.

6 What you should be able to say

- The loop is observe, decide, act, feedback.
- A fluent email with no tool call is not an agent.
- Name one job for an LLM-based agent and one job for an RL-based agent.
- Say why a chatbot is a poor inventory controller.
- Name the fifth managerial question: the human checkpoint.

Quiz 1 covers this session. It is the multiple-choice quiz on Canvas, due one hour before Session 2. A late quiz is not accepted except with an excused absence.